

Predicción de puntuación y capitalización en texto normalizado



Carolina Kung, Miguel Ávila Carrero, Micaela Pego, Malena Sanz
Facultad de Ciencias Exactas y Naturales (UBA) - Aprendizaje Automático II - 2025



Introducción

Los sistemas de reconocimiento automático del habla (ASR) suelen producir **texto plano**, en minúsculas y sin puntuación. De esta manera se pierde el sentido original de la oración y se vuelve más complicado utilizar ese texto en aplicaciones, por ejemplo en subtitulado automático.

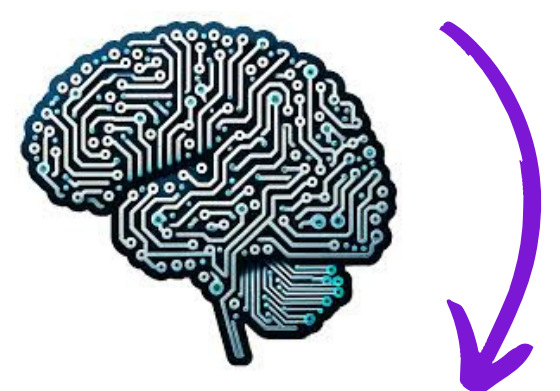
Objetivo

En este trabajo abordamos el problema de **predecir la puntuación inicial** (“ ”, “¿”), **puntuación final** (“ ”, “.”, “,”, “?”) y tipo de **capitalización** para cada palabra de una oración dada (ninguna mayúscula, sólo primera letra mayúscula, capitalización parcial o total). Para eso implementamos tres modelos: Random Forest, RNN y RNN bidireccional.

1. Preguntas clave

- ¿Conviene compartir pesos entre tareas?
- ¿Qué clase le costó más a qué modelo?
- ¿Cómo se relaciona con su arquitectura?

"cuándo salimos para la uba..."



"¿Cuándo salimos para la UBA?...?"

2. Dataset y preprocesamiento

Fuente: Dataset CC100. Es un corpus de texto a gran escala que contiene datos en 100 idiomas diferentes, 14.3 GB de instancias de texto en español.

Filtrado y preprocesado: Limpieza de instancias mal escritas. Falta de mayúsculas, puntuación o cantidad de símbolos de interrogación dispar, signos no permitidos, texto completamente en mayúsculas, etc.

Tokenización: BERT - base - multilingual - cased

equipo microsoft $\xrightarrow{\text{BERT Tokenizer}}$ equipo micro ##so ##ft $\xrightarrow{\text{Embeddings estáticos dim 768}}$ emb1 emb2 emb3 emb4

Desbalanceo de clases: Naturalmente las clases de cada tarea estaban desbalanceadas. Por ejemplo, un espacio en blanco es mucho mas frecuente que un punto.

3. Random Forest

Extracción de atributos: Seleccionamos manualmente 15 características que consideramos importantes como por ejemplo:

hola cómo **estás** $\xrightarrow{\text{embedding}}$ (1.2, 0.3, ..., 0.8) $\xrightarrow{\text{token position: 3}}$ dist(cómo, estás): 0.4
 $\xrightarrow{\text{mean embedding: 0.75}}$

Construcción del modelo: hicimos un RF separado para cada problema. Esto implica que aquí **no se compartieron pesos entre tareas**. Fue muy importante **balancear** las clases dándoles un peso inversamente proporcional a su frecuencia en la función de pérdida.

Análisis de resultados: exploramos métricas como F1 score y matrices de confusión pero también obtuvimos la importancia de los atributos.

4. RNN estándar y bidireccional

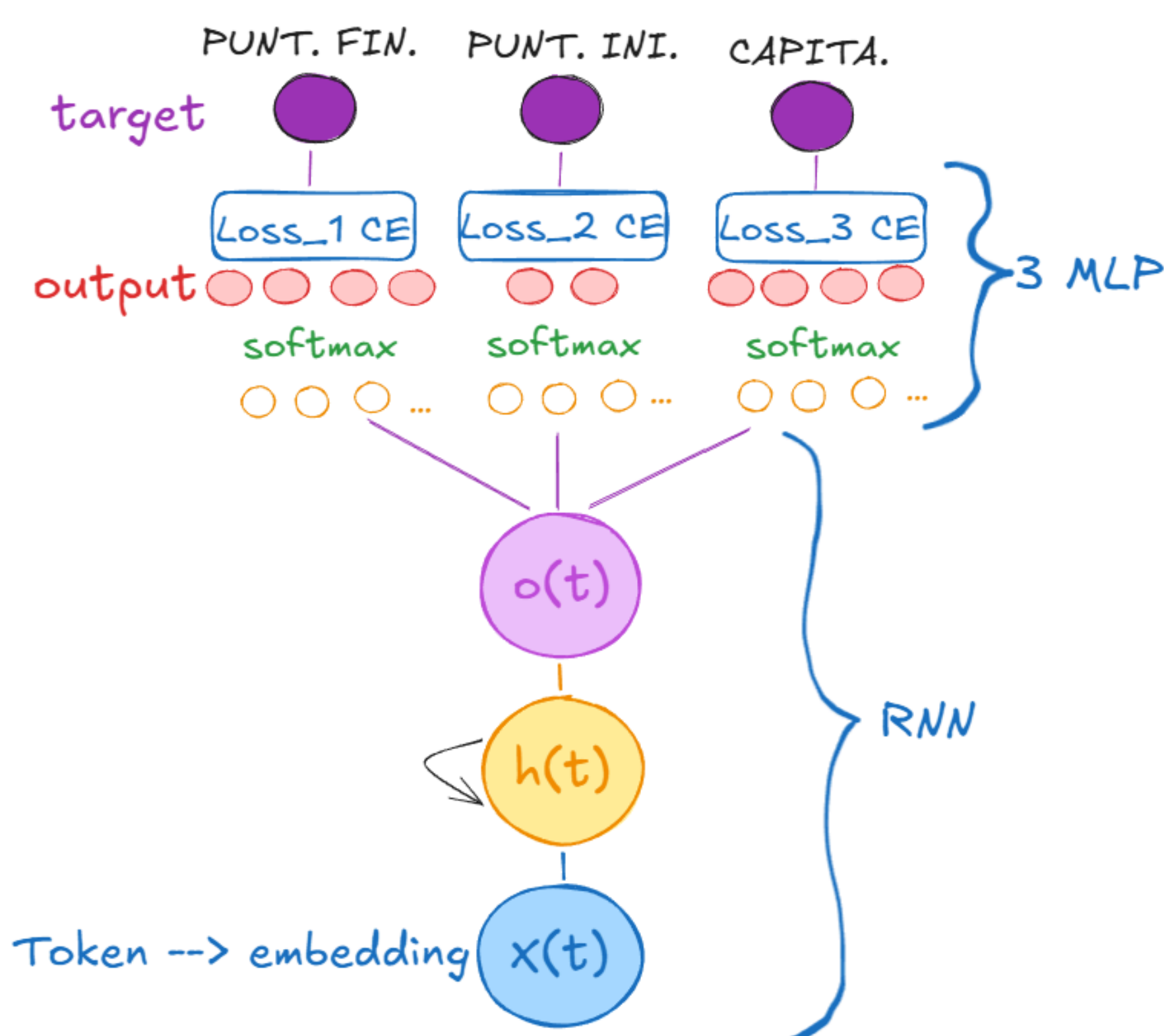
Entrada: Cada instancia es una secuencia de embeddings. Usamos **padding** para que todas las secuencias tengan la misma longitud (dentro de cada batch).

Arquitectura: Multi task learning. Una sola red para todas las clases de todas las tareas.

- Red recurrente de tipo **LSTM** (unidireccional y luego bidireccional) con dos capas
- La RNN se conecta a **tres fully connected layers independientes** que generan tres salidas

Salida: Se obtienen tres salidas. Cada salida tiene las probabilidades de las clases para cada problema.

Desbalanceo de clases: nuevamente usamos pesos inversamente proporcionales a la frecuencia y agregamos un **suavizado** sobre las etiquetas objetivo para evitar una “sobreconfianza” del modelo sobre la clase mayoritaria.



5. Resultados

Feature importance de RF

Puntuación inicial	Puntuación final	Capitalización
• Token position norm	• Token position norm	• Emb cos distance
• Token position	• Token distance next	• Is numeric
• Emb distance prev	• Token distance prev	• Emb std

Comparación de F1 macro average entre modelos

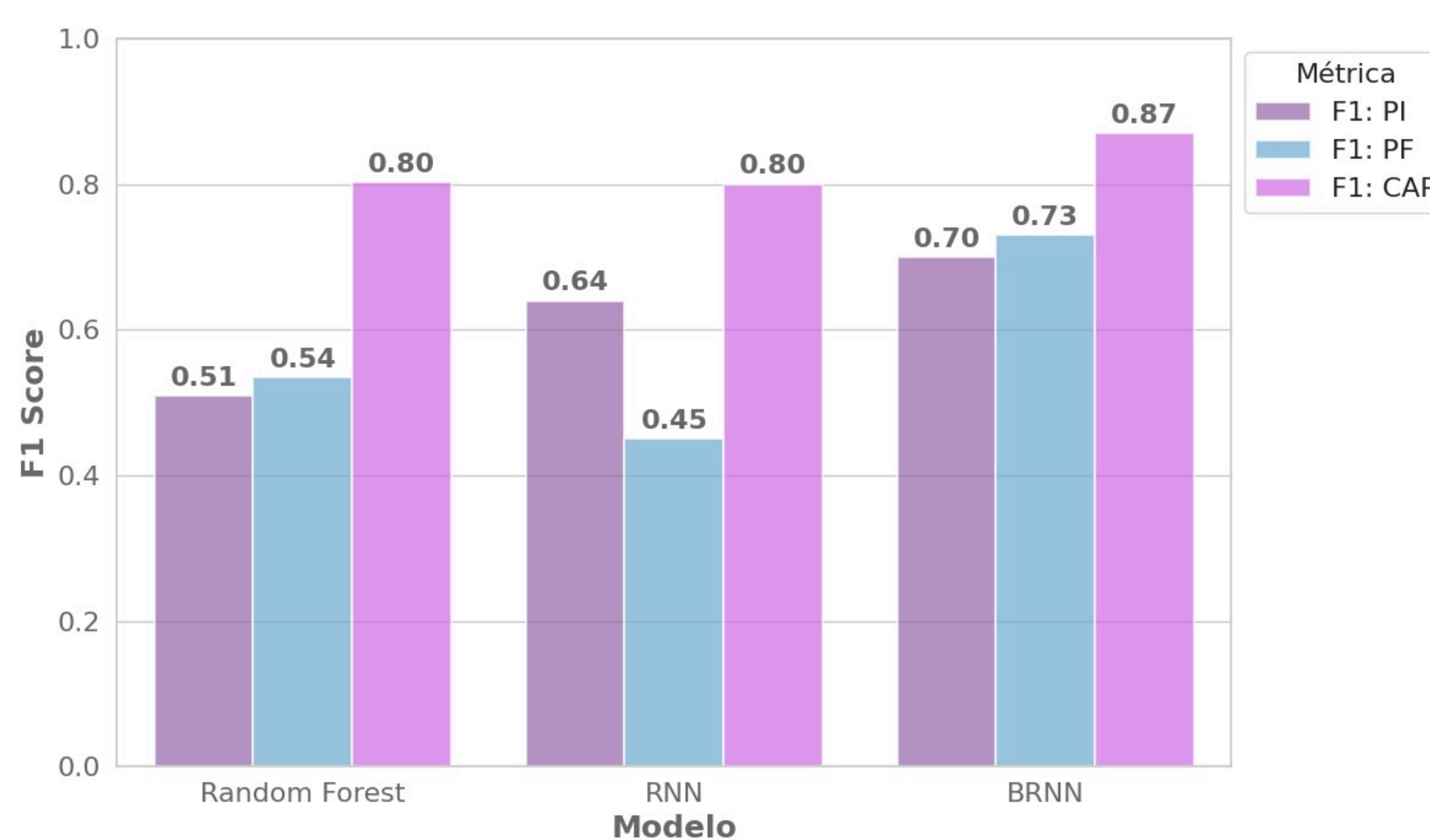


Figura 1: F1 score macro average de cada modelo. Cada barra representa el F1 score promediado sobre todas las clases de cada problema.

6. Análisis

- En capitalización los tres modelos tienen performance similar. Esto puede deberse a que este problema depende menos del contexto que el problema de puntuación y tiene características más determinantes (por ej, ser la primera palabra de la oración). En este sentido, RNN y BiRNN no ganan demasiado porque no se necesita mucha información secuencial.
- Para el problema de puntuación, la BiRNN tiene mejor score. Esto coincide con lo que esperábamos, ya que el problema de puntuación depende fuertemente tanto del contexto pasado como del futuro. Este modelo usa información de las palabras posteriores explícitamente y comparte pesos entre los problemas, que están claramente correlacionados.
- La RNN funcionó peor para la predicción de puntuación final que para los otros problemas. Creemos que esto es porque la puntuación final depende en gran parte de lo que ocurre después del token, y al no contar con esa información, la performance de la RNN se ve muy perjudicada. En cambio, la BiRNN utiliza esta información explícitamente y el RF de manera más indirecta con atributos como “¿es el último token?” o la distancia coseno al embedding de la palabra siguiente.